

Module 3.4

Constraint satisfaction

Dr. Nirmala Baloorkar

Asst. Prof.

Dept of Computer Engg.



SOMAIYA
VIDYAVIHAR UNIVERSITY

K J Somaiya College of Engineering



Outline

- Constraint Satisfaction
- Example
- Inferences in CSP,
- CSP backtracking algorithm

Constraint satisfaction problems (CSPs)

- **Consist of three components**
 - A set of **variables** X_i
 - A set of **Domain** D_i
 - A set of **Constraints** C_i *Specifiying allowable combination of values subsets of variables*
- **Our task is to assign a value to each variable so that we fulfill all the constraints.**

Constraint satisfaction problems (CSPs)

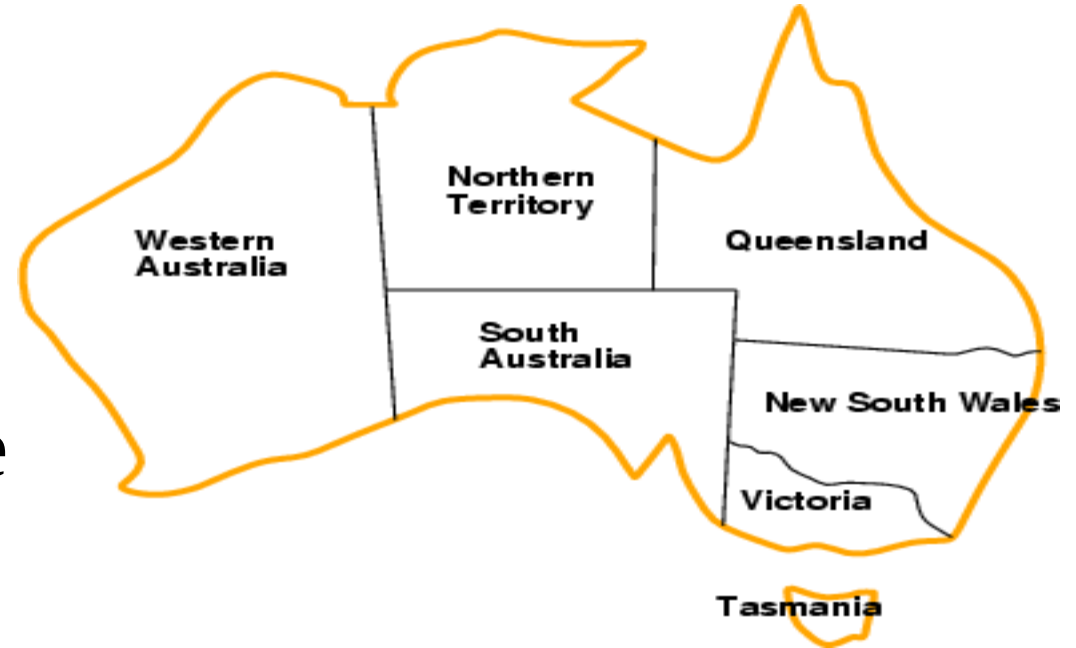
- It consists of the following:
 - A finite set of **variables** which stores the solution ($V = \{V_1, V_2, V_3, \dots, V_n\}$)
 - A set of **discrete** values known as **domain** from which the solution is picked ($D = \{D_1, D_2, D_3, \dots, D_n\}$)
 - A finite set of **constraints** ($C = \{C_1, C_2, C_3, \dots, C_n\}$)

Uninformed (“Blind”) Search Strategies

- No information is available other than
 - The current state
 - ♦ Its parent (perhaps complete path from initial state)
 - ♦ Its operators (to produce successors)
 - The goal test
 - The current path cost (cost from start state to current state)

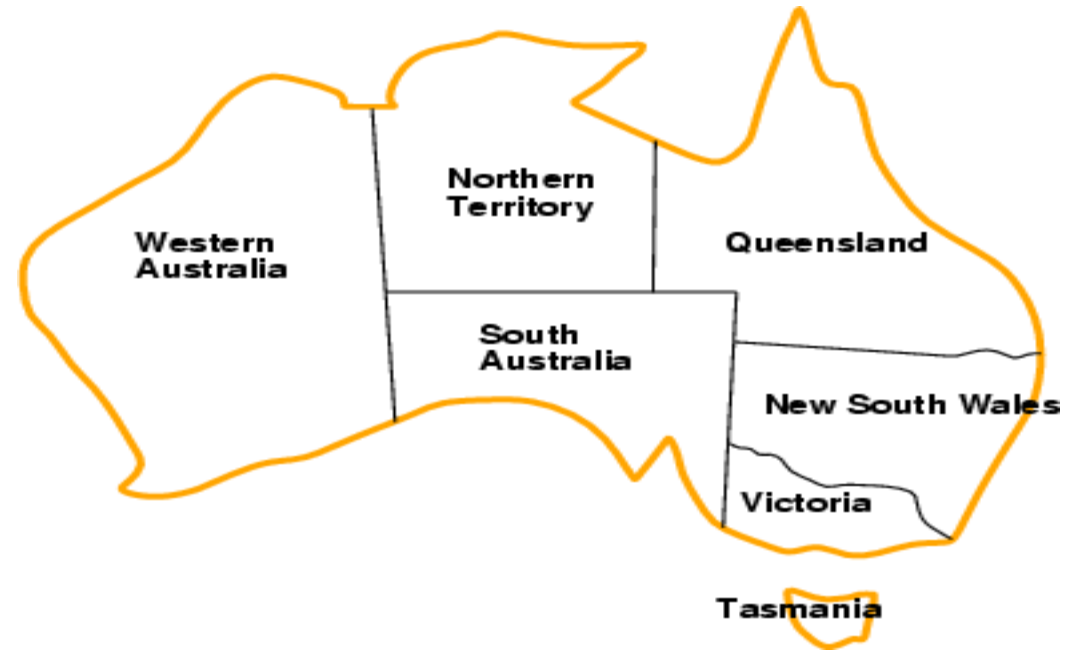
Example: Map-Coloring

Problem Statement: Colour each region either red, green or blue in such a way that no neighbouring regions have the same colour.

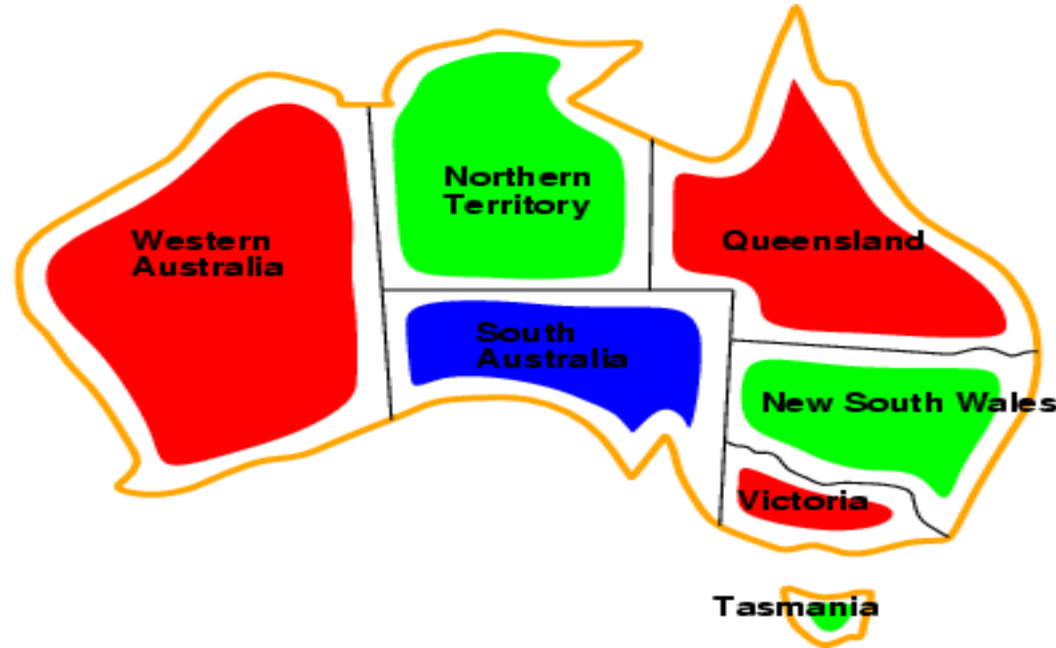


Example: Map-Coloring

- **Variables** WA, NT, Q, NSW, V, SA, T
- **Domains** $D_i = \{\text{red, green, blue}\}$
 - **Constraints:** adjacent regions must have different colors
 - e.g., $WA \neq NT$, or (WA, NT) in $\{(\text{red, green}), (\text{red, blue}), (\text{green, red}), (\text{green, blue}), (\text{blue, red}), (\text{blue, green})\}$



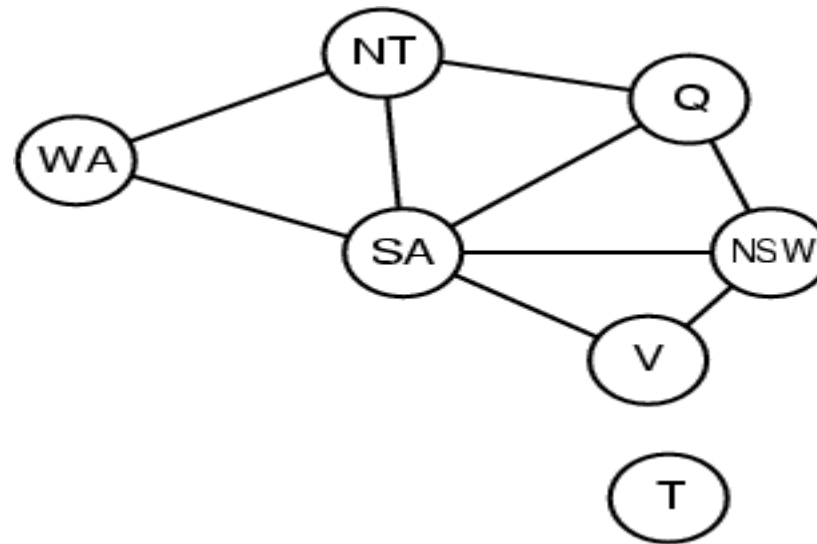
Example: Map-Coloring



- **Solutions** are **complete** and **consistent** assignments,
 - e.g., WA = red, NT = green, Q = red, NSW = green, V = red, SA = blue, T = green

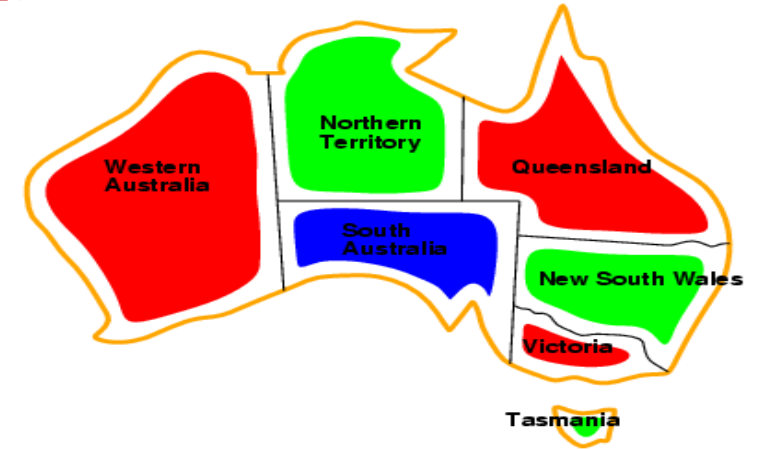
Constraint graph

- **Binary CSP:** each constraint relates two variables
- **Constraint graph:** nodes are variables, arcs are constraints



Varieties of constraints

- **Unary** constraints involve a single variable,
 - e.g., $SA \neq \text{green}$
- **Binary** constraints involve pairs of variables,
 - e.g., $SA \neq WA$
- **Higher-order** constraints involve 3 or more variables,
 - e.g., crypt arithmetic column constraints



Crypt - arithmetic Problems

- Also called as alphabetic/verbal arithmetic/word addition
- Mathematical game where a mathematical equation is presented where digits in the equation are replaced by letters.
- E.g. **TO + GO = OUT.**

Crypt - arithmetic Problems

- **Unique Letter-Digit Mapping:**
 - Each letter represents a unique digit ranging from 0 to 9.
 - A digit cannot correspond to more than one letter.
- **Rules for Leading Digits:**
 - Numbers cannot begin with the digit zero (e.g., 0123 is incorrect, while 123 is correct).
- **Carry in Addition:**
 - In addition problems, any carryover is strictly limited to the value of 1.
- There must be only one solution .

Example $TO + GO = OUT$

Step 1 Identify the variables

- The equation has letters: T, O, G, U.
- Each letter represents a unique digit (0-9).
- The digits must satisfy the equation, and no number can start with 0.

Step 2 Analyse the equation structure

- **TO** and **GO** are two-digit numbers.
- **OUT** is a three-digit number.
- T + G is generating O is carry
 - value of O is 1

$$\begin{array}{r} \text{T O} \\ + \text{G O} \\ \hline \text{O U T} \end{array}$$

Example $TO + GO = OUT$ (continued)

$$\begin{array}{r} T\ O \\ +\ G\ O \\ \hline O\ U\ T \end{array}$$

Step 3 Determine the Value of T

- Substitution of value of O is 1
- The digits must satisfy the equation, and no number can start with 0.

Step 4 Analyze G and U.

$$\begin{array}{r} T\ 1 \\ +\ G\ 1 \\ \hline 1\ U\ T \end{array}$$

$$\begin{array}{r} 2\ 1 \\ +\ G\ 1 \\ \hline 1\ U\ 2 \end{array}$$

Example TO + GO = OUT (continued)

$$\begin{array}{r} 21 \\ + G1 \\ \hline 1U2 \end{array}$$

Step 4 Analyse G and U.

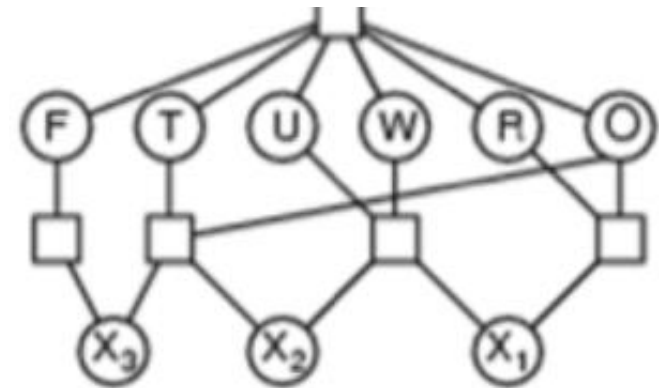
Now, $2 + G > 10$ (as its resulting a carry 1 on next)

- Now, possible values of G to get 1 carry at next step is – $\{G = 8 \text{ or } 9\}$
 - If G is 9 then – $U = 2 + 9 = 11$
 - So value of U becomes 1 and 1 goes to carry
 - Now, value of O is already 1 so U value can not be 1 also. **(game rule)**
- Now, the value of G has to be 8, lets verify this also
 - $U = 2 + 8 = 10$
 - So value of U becomes 0, there is no conflict.
- So final values are – $T = 2, O = 1, G = 8, U = 0$

Example

- **Variables:** F, T, U, W, R, O $X_1 X_2 X_3$
- **Domains:** $\{0, 1, 2, 3, 4, 5, 6, 7, 8, 9\}$
- **Constraints:** $\text{Alldiff}(F, T, U, W, R, O)$
 - $O + O = R + 10 \cdot X_1$
 - $X_1 + W + W = U + 10 \cdot X_2$
 - $X_2 + T + T = O + 10 \cdot X_3$
 - $X_3 = F, T \neq 0, F \neq 0$

$$\begin{array}{r} T \ W \ O \\ + \ T \ W \ O \\ \hline F \ O \ U \ R \end{array}$$



Exercise

$$\begin{array}{r} \text{S E N D} \\ + \text{M O R E} \\ \hline = \text{M O N E Y} \end{array} \quad \begin{array}{r} \text{E A T} \\ + \text{T H A T} \\ \hline = \text{A P P L E} \end{array} \quad \begin{array}{r} \text{B A S E} \\ + \text{B A L L} \\ \hline = \text{G A M E S} \end{array}$$

Real-world CSPs

- Assignment problems
 - e.g., who teaches what class
 - Timetabling problems
 - e.g., which class is offered when and where?
 - Transportation scheduling
 - Factory scheduling
- Notice that many real-world problems involve real-valued variables

Backtracking search

- Variable assignments are **commutative**, i.e.,
[WA = red then NT = green] same as [NT = green then WA = red]
- Only need to consider assignments to a single variable at each node
→ $b = d$ and there are d^n leaves
- Depth-first search for CSPs with single-variable assignments is called **backtracking** search
- Backtracking search is the basic uninformed algorithm for CSPs
- Can solve n -queens for $n \approx 25$

Backtracking search

```
function BACKTRACKING-SEARCH(csp) returns a solution, or failure
  return RECURSIVE-BACKTRACKING( {}, csp)

function RECURSIVE-BACKTRACKING(assignment, csp) returns a solution, or failure
  if assignment is complete then return assignment
  var ← SELECT-UNASSIGNED-VARIABLE( Variables[csp], assignment, csp)
  for each value in ORDER-DOMAIN-VALUES(var, assignment, csp) do
    if value is consistent with assignment according to Constraints[csp] then
      add { var = value } to assignment
      result ← RECURSIVE-BACKTRACKING(assignment, csp)
      if result ≠ failure then return result
      remove { var = value } from assignment
  return failure
```

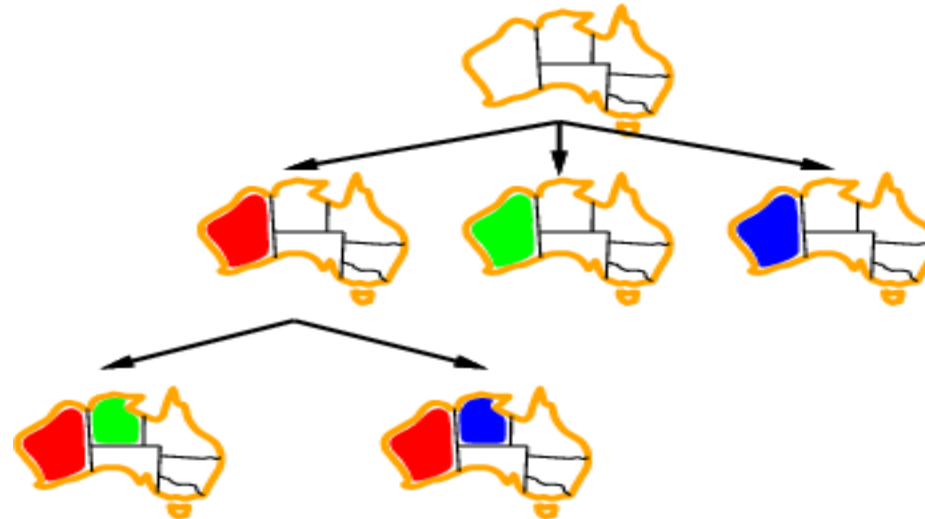
Backtracking example



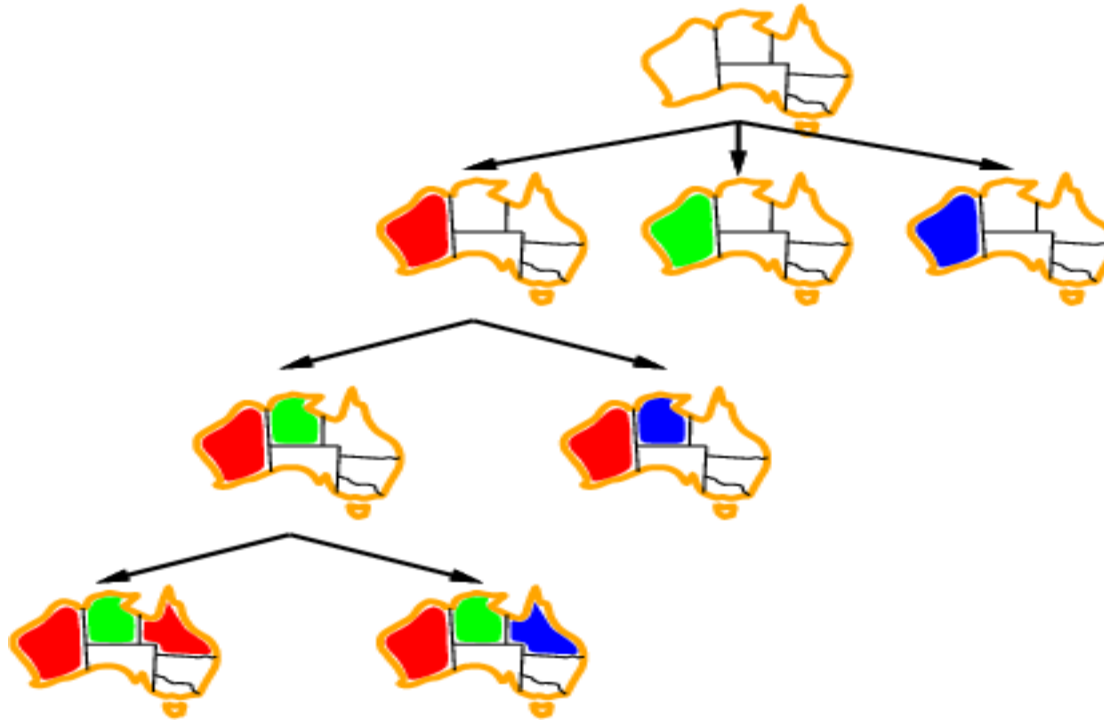
Backtracking example



Backtracking example



Backtracking example

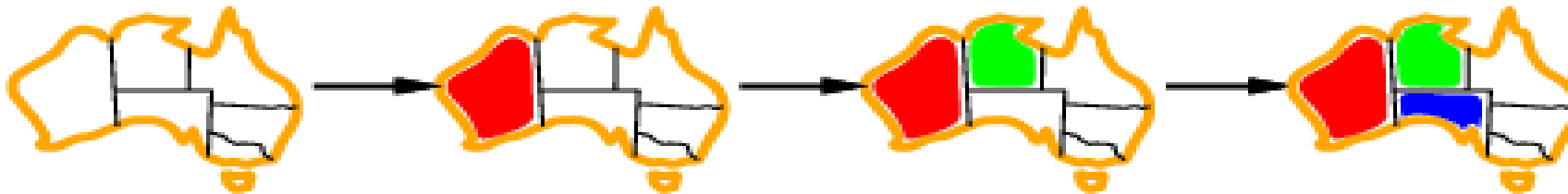


Improving backtracking efficiency

- **General-purpose** methods can give huge gains in speed:
 -
 - Which variable should be assigned next?
 - In what order should its values be tried?
 - Can we detect inevitable failure early?

Most constrained variable

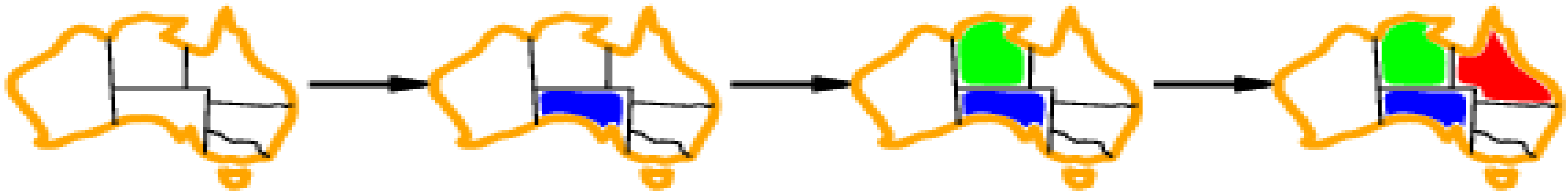
- Most constrained variable:
choose the variable with the fewest legal values



- a.k.a. **minimum remaining values (MRV)** heuristic
-

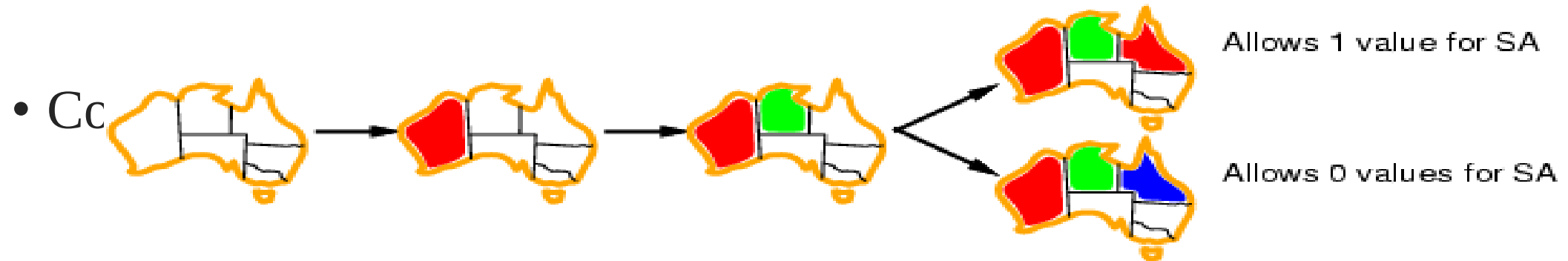
Most constraining variable

- Tie-breaker among most constrained variables
- Most constraining variable:
 -
 - choose the variable with the most constraints on remaining variables
 -



Least constraining value

- Given a variable, choose the least constraining value:
- - the one that rules out the fewest values in the remaining variables
 -



Forward checking

- Idea:

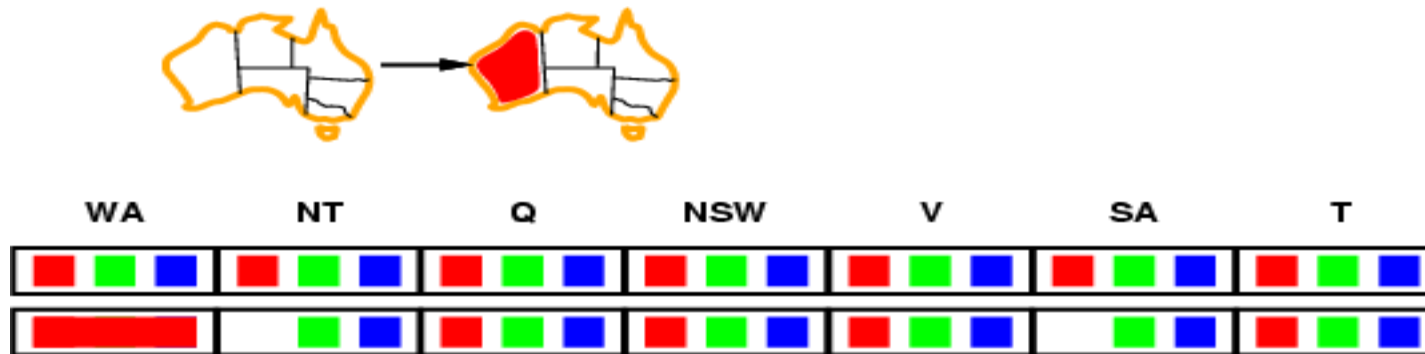
- Keep track of remaining legal values for unassigned variables
- Terminate search when any variable has no legal values
-



Forward checking

- Idea:

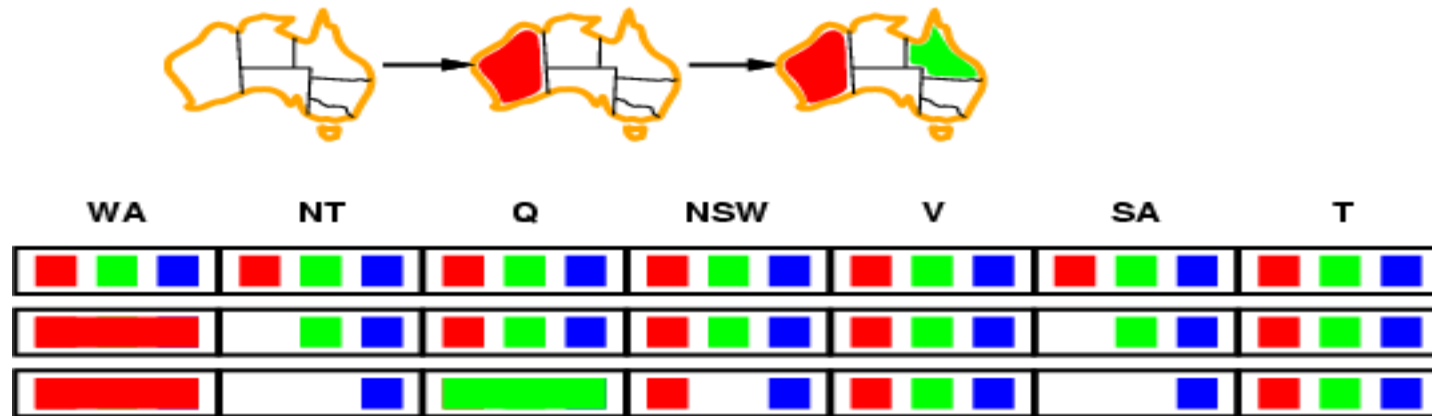
- Keep track of remaining legal values for unassigned variables
- Terminate search when any variable has no legal values
-



Forward checking

- Idea:

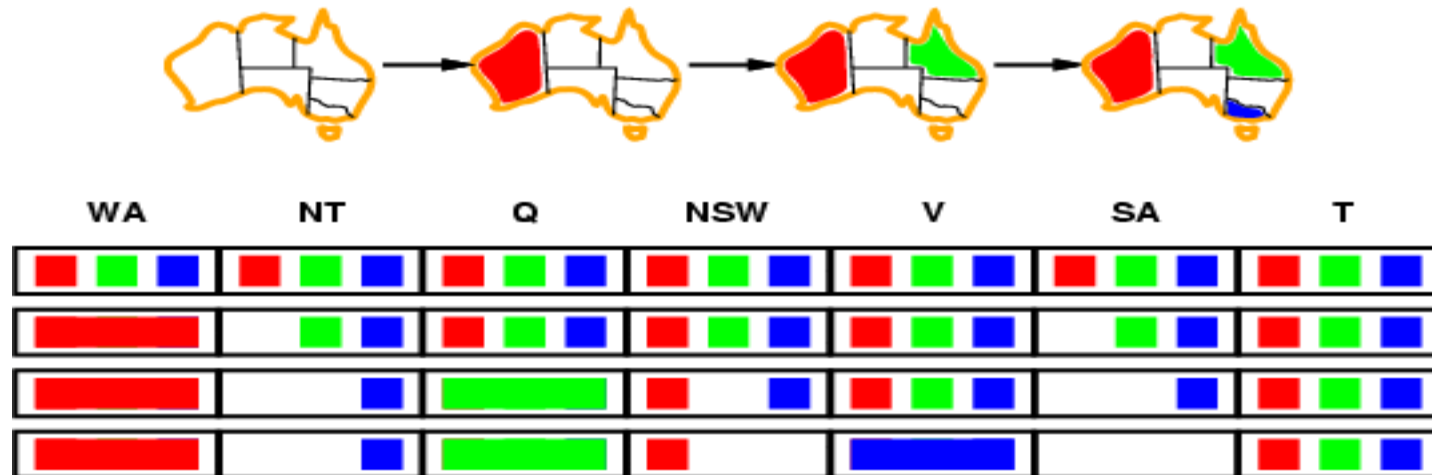
- Keep track of remaining legal values for unassigned variables
- Terminate search when any variable has no legal values
-



Forward checking

- Idea:

- Keep track of remaining legal values for unassigned variables
- Terminate search when any variable has no legal values
-



Constraint propagation

- Forward checking propagates information from assigned to unassigned variables, but doesn't provide early detection for all failures:

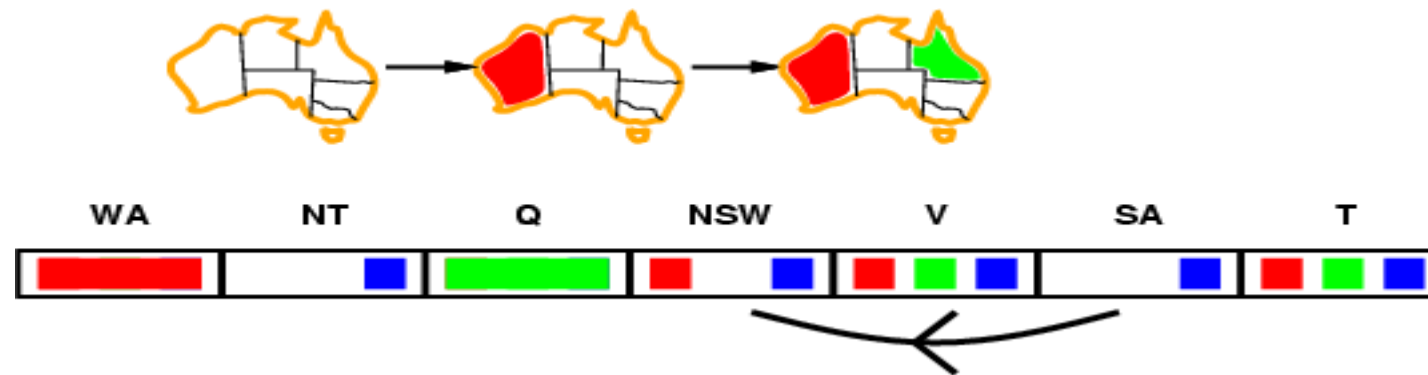


WA	NT	Q	NSW	V	SA	T

- NT and SA cannot both be blue:
- **Constraint propagation** repeatedly enforces constraints locally

Arc consistency

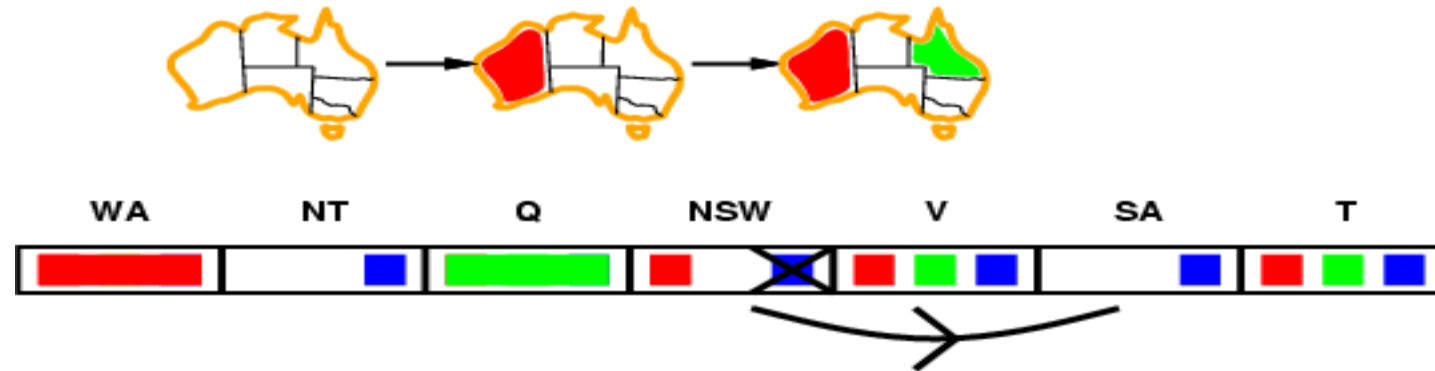
- Simplest form of propagation makes each arc **consistent**
- $X \rightarrow Y$ is consistent iff
- for **every** value x of X there is **some** allowed y



Arc consistency

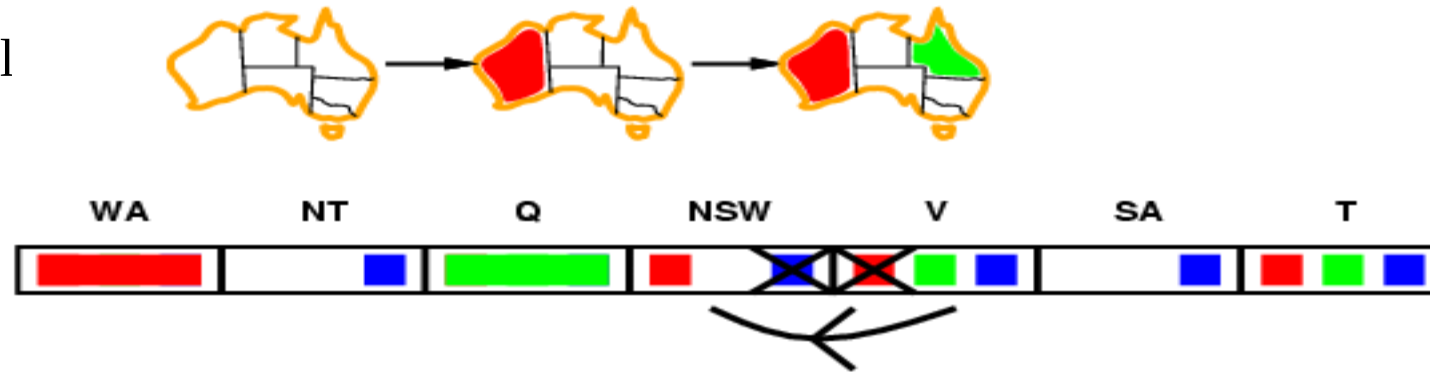
- Simplest form of propagation makes each arc consistent
- $X \rightarrow Y$ is consistent iff
- ---

for every value x of X there is some allowed y



Arc consistency

- Simplest form of propagation makes each arc **consistent**
- $X \rightarrow Y$ is consistent iff
- for **every** val



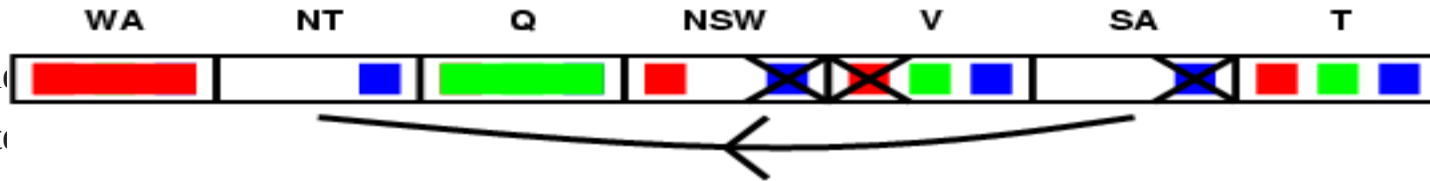
- If X loses a value, neighbors of X need to be rechecked
-

Arc consistency

- Simplest form of propagation makes each arc **consistent**
- $X \rightarrow Y$ is consistent iff
- for **every** value x of X there is **some** allowed y



- If X loses a value, no
- Arc consistency det



- Can be run as a preprocessor or after each assignment

Arc consistency algorithm AC-3

```
function AC-3( csp) returns the CSP, possibly with reduced domains
inputs: csp, a binary CSP with variables  $\{X_1, X_2, \dots, X_n\}$ 
local variables: queue, a queue of arcs, initially all the arcs in csp

while queue is not empty do
     $(X_i, X_j) \leftarrow \text{REMOVE-FIRST}(\textit{queue})$ 
    if RM-INCONSISTENT-VALUES( $X_i, X_j$ ) then
        for each  $X_k$  in NEIGHBORS[ $X_i$ ] do
            add  $(X_k, X_i)$  to queue



---


function RM-INCONSISTENT-VALUES(  $X_i, X_j$ ) returns true iff remove a value
    removed  $\leftarrow$  false
    for each  $x$  in DOMAIN[ $X_i$ ] do
        if no value  $y$  in DOMAIN[ $X_j$ ] allows  $(x, y)$  to satisfy constraint( $X_i, X_j$ )
            then delete  $x$  from DOMAIN[ $X_i$ ]; removed  $\leftarrow$  true
    return removed
```

- Time complexity: $O(n^2d^3)$



SOMAIYA
VIDYAVIHAR UNIVERSITY

K J Somaiya College of Engineering

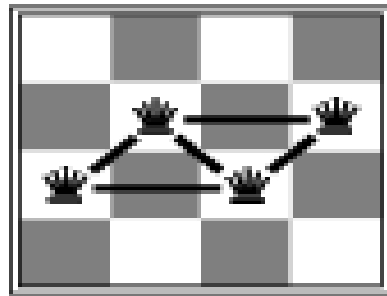


Local search for CSPs

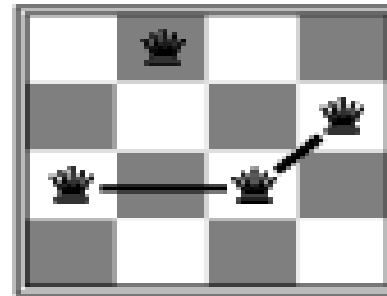
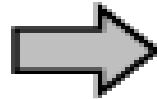
- Hill-climbing, simulated annealing typically work with "complete" states, i.e., all variables assigned
- To apply to CSPs:
 - allow states with unsatisfied constraints
 - operators **reassign** variable values
- Variable selection: randomly select any conflicted variable
- Value selection by **min-conflicts** heuristic:
 - choose value that violates the fewest constraints
 - i.e., hill-climb with $h(n)$ = total number of violated constraints

Example: 4-Queens

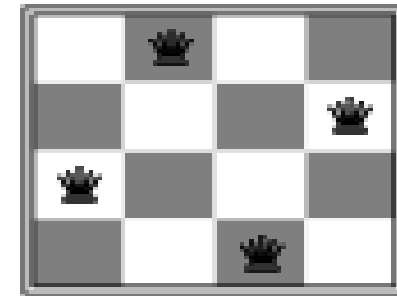
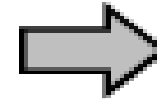
- **States:** 4 queens in 4 columns ($4^4 = 256$ states)
- **Actions:** move queen in column
- **Goal test:** no attacks
- **Evaluation:** $h(n)$ = number of attacks



$h = 5$



$h = 2$



$h = 0$

- Given random initial state, can solve n -queens in almost constant time for arbitrary n with high probability (e.g., $n = 10,000,000$)

Summary

- CSPs are a special kind of problem:
 - states defined by values of a fixed set of variables
 - goal test defined by constraints on variable values
- Backtracking = depth-first search with one variable assigned per node
- Variable ordering and value selection heuristics help significantly
- Forward checking prevents assignments that guarantee later failure
- Constraint propagation (e.g., arc consistency) does additional work to constrain values and detect inconsistencies
- Iterative min-conflicts is usually effective in practice

THANK YOU



SOMAIYA
VIDYAVIHAR UNIVERSITY

K J Somaiya College of Engineering

